

Introduction to MATLAB

99.500 Applied Mathematics for Engineering
From Independent Study by Tan Xue Feng (2024)

Learning Outcomes

- Gain familiarity with MATLAB capabilities
- Learn syntax to perform basic calculations in MATLAB
- Solve ODEs using MATLAB's built-in functions

MATLAB for numerical computation, visualization and programming

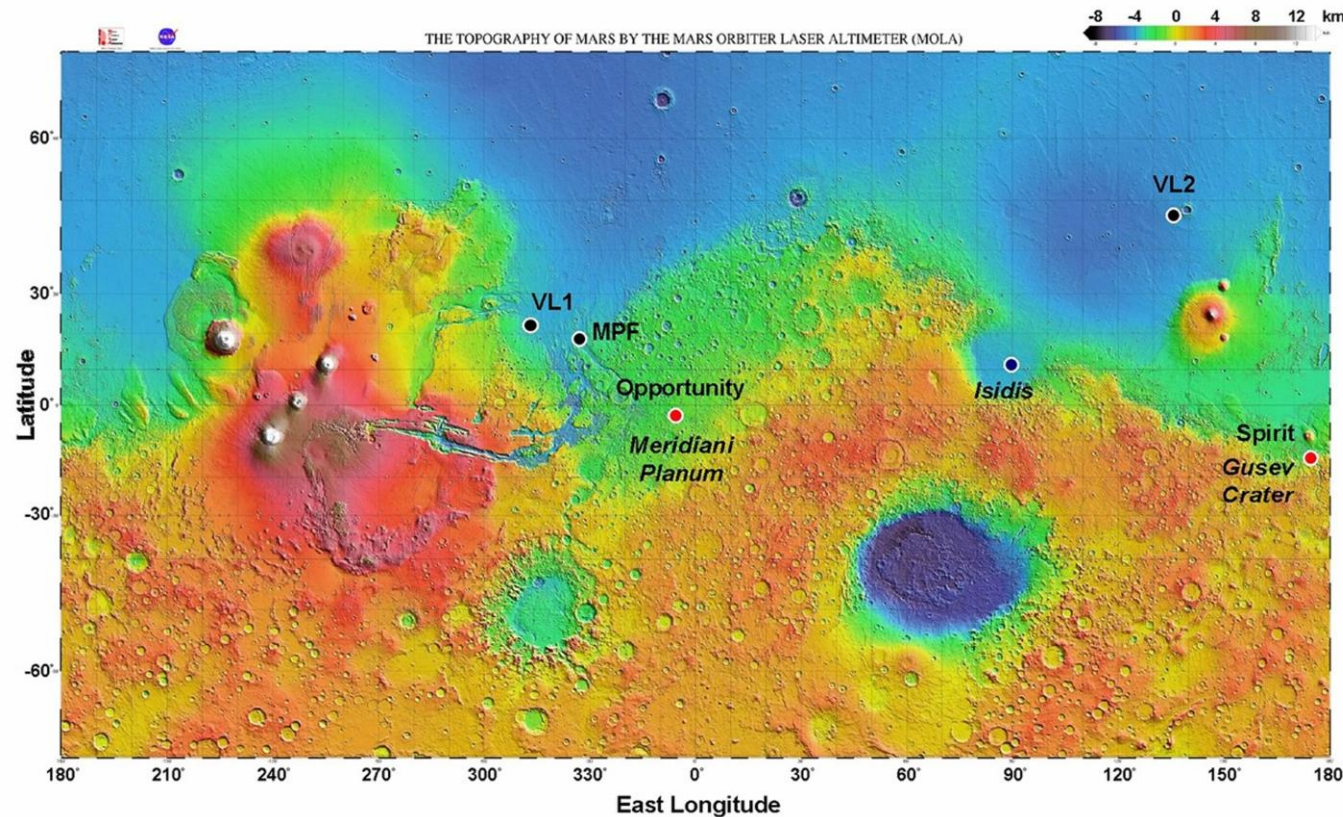
- MATLAB (Matrix LABoratory) is a high-level programming language and interactive environment for numerical computation, visualization and programming.
- Developed in the late 1970s by Cleve Moler, a professor at the university of New Mexico
 - Started as a learning tool for linear algebra and numerical analysis without learning Fortran
 - Later evolved from an academic tool to a commercial software product
- Key features:
 - Numerical computing with matrices and arrays
 - Data visualization and graphics
 - Built-in functions and toolboxes
 - Interactive environment for iterative exploration

Mars rover landing: navigation, data-analysis, EDL using MATLAB (2004)



Mars rover landing: navigation, data-analysis, EDL using MATLAB (2004)

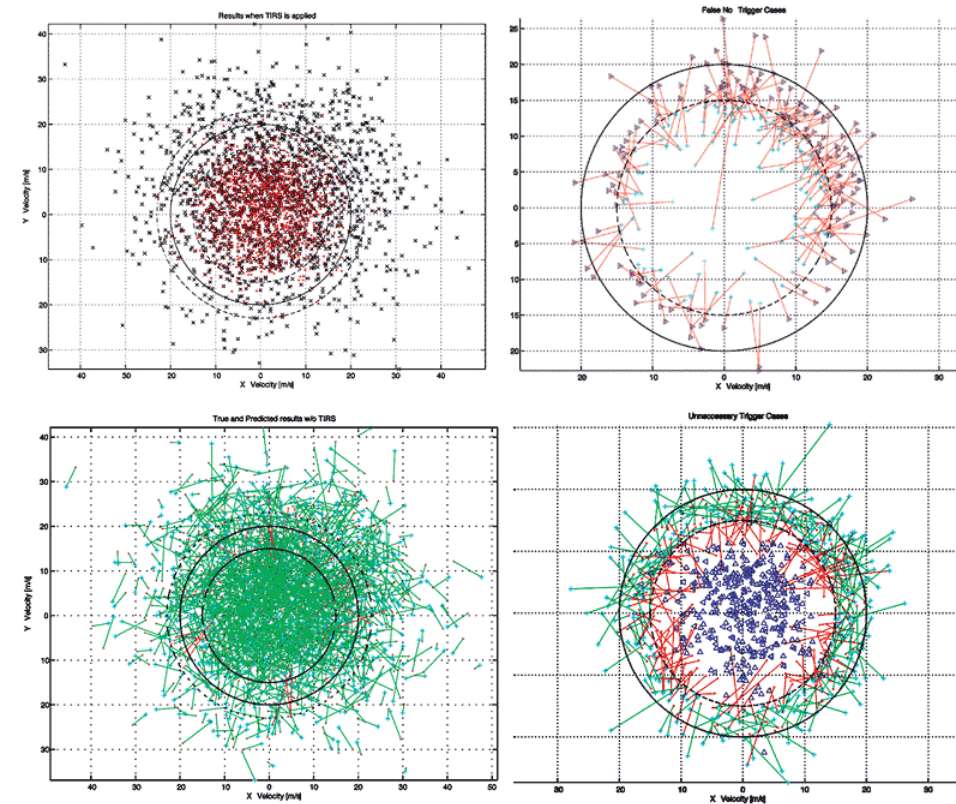
- Jet Propulsion Laboratory (JPL) engineers used MATLAB and Simulink for navigation, data-analysis and entry, descent and landing (EDL) system design
 - Built a simulation tool in MATLAB that accounts for Martian surface to prove that chosen landing site can be reached



Meridiani Planum and Gusev Crater – landing sites chosen for Opportunity and Spirit respectively

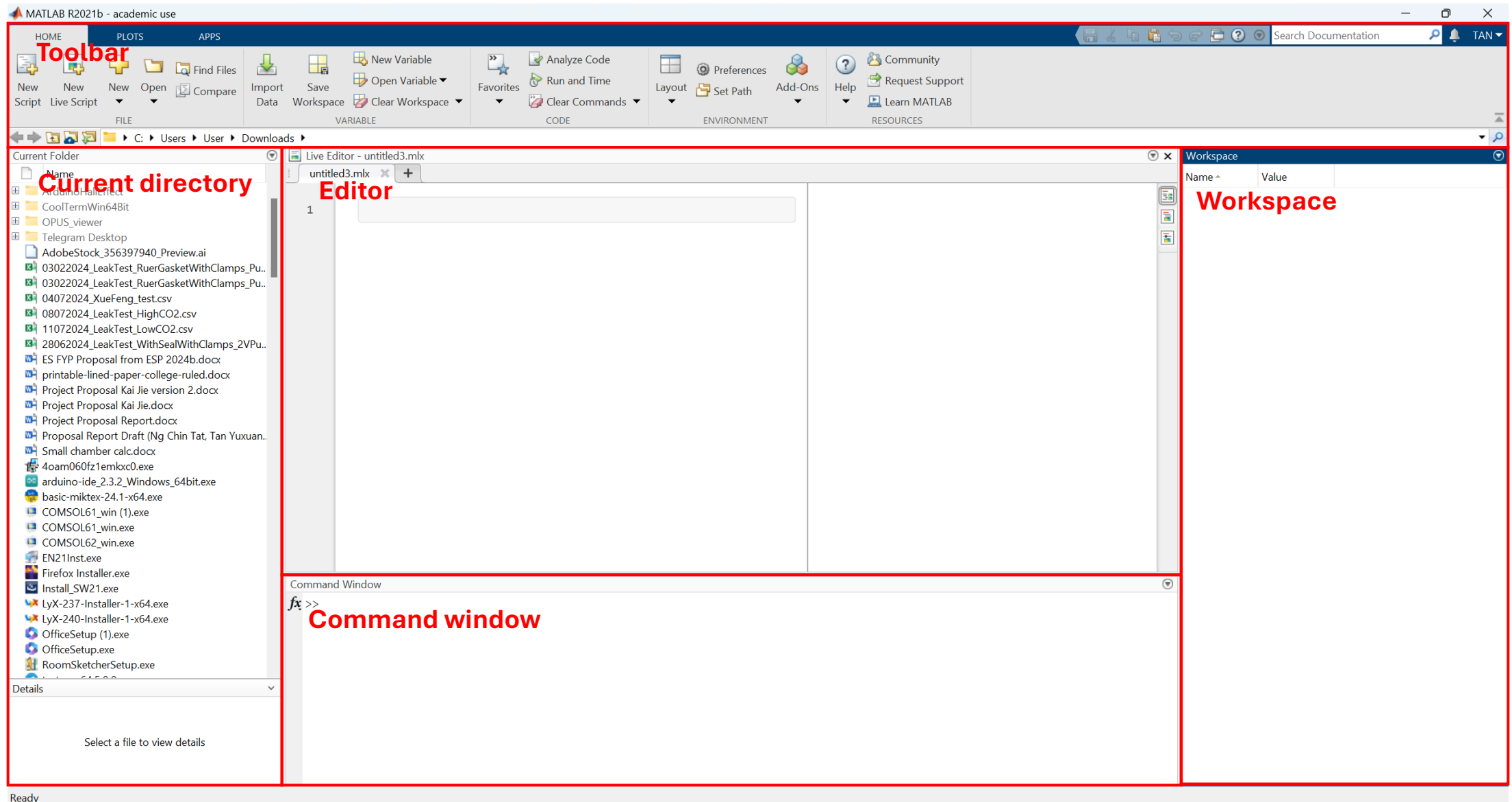
Mars rover landing: navigation, data-analysis, EDL using MATLAB (2004)

- Entry, descent and landing (EDL) system ensures landing craft would withstand disturbances from Martian atmosphere kilometres above landing site and determine how on-board systems react
- Statistical program development
 - Transverse Impulse Rocket System (TIRS)
 - Last-second altitude correction to protect landing craft air airbags from wind and gusts
 - Descent Image Motion Estimation System (DIMES)
 - Pictures to detect prevailing wind and residual trajectory velocity



MATLAB predictions on how onboard systems would react under different external conditions

MATLAB provides an intuitive and interactive environment



1. Select 'text' 2. Select type of text

3. Type text

This is a title for my document

1. Select 'code'

This is a title for the MATLAB document

2. Type code

Numerical computing and visualization with matrices and arrays

Matrix operations

```
% Define matrices A and B
A = [1, 2, 3; 4, 5, 6; 7, 8, 9];
B = [3, 4, 5; 6, 7, 8; 9, 10, 11];

% Scalar multiplication
C = A * 2

% Matrix multiplication
D = A * B

% Element wise multiplication
E = A .* B
```

```
C = 3x3
     2     4     6
     8    10    12
    14    16    18

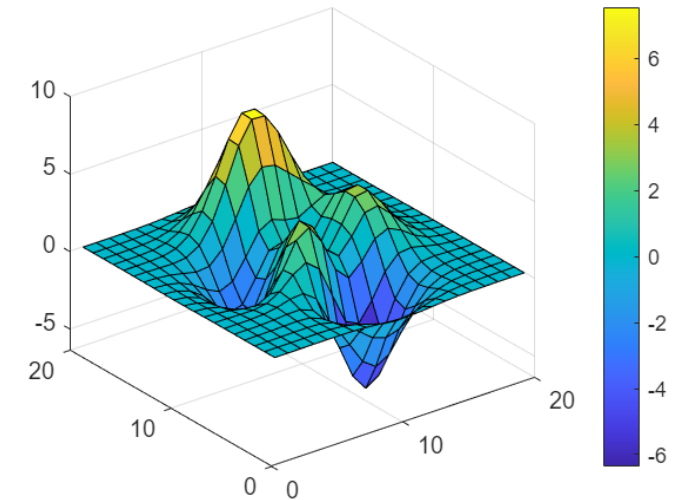
D = 3x3
    42    48    54
    96   111   126
   150   174   198

E = 3x3
     3     8    15
    24    35    48
    63    80    99
```

3D visualisation of matrix

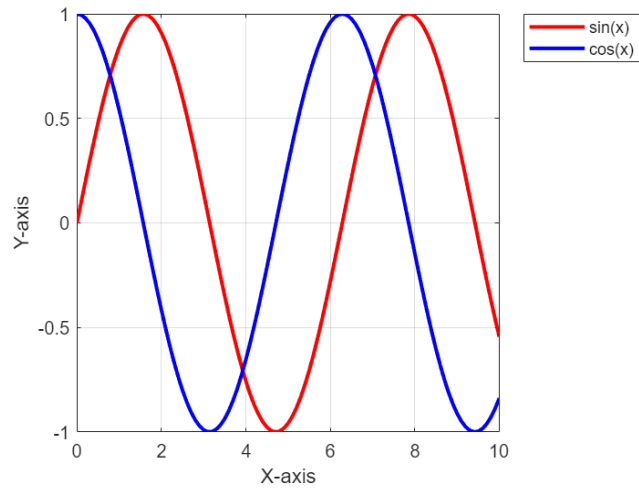
```
randMatrix = 20x20
    0.0001    0.0003    0.0011    0.0030    0.0051 ...
    0.0002    0.0009    0.0031    0.0079    0.0119
    0.0004    0.0020    0.0073    0.0183    0.0260
    0.0007    0.0036    0.0137    0.0371    0.0610
    0.0006    0.0040    0.0186    0.0608    0.1336
   -0.0011   -0.0015    0.0072    0.0562    0.2012
   -0.0061   -0.0206   -0.0479   -0.0546    0.0870
   -0.0153   -0.0577   -0.1659   -0.3412   -0.4162
   -0.0264   -0.1044   -0.3221   -0.7512   -1.2440
   -0.0348   -0.1404   -0.4464   -1.0928   -1.9840
    ...
    ...
    ...
```

20 x 20 matrix

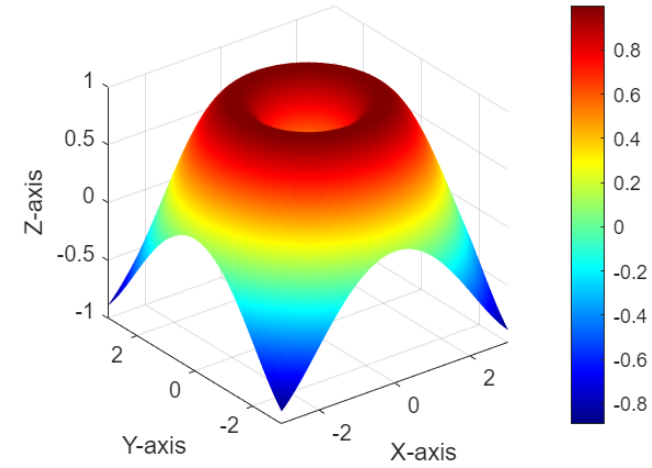


Surface plot of matrix

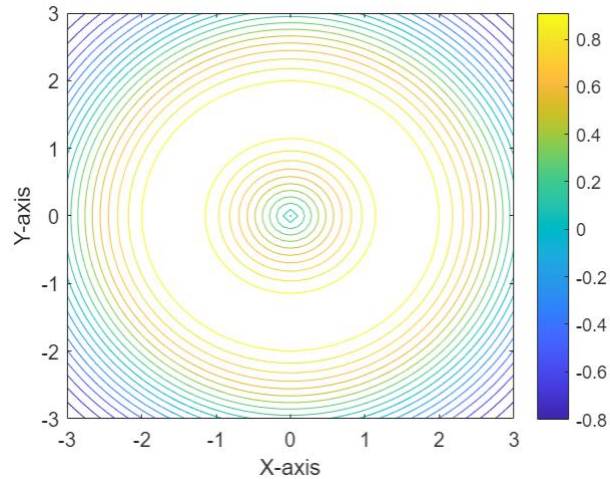
Data visualization with graphics



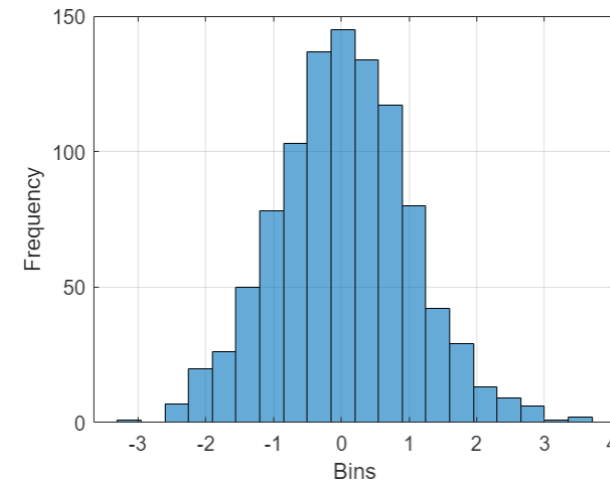
2D plot



3D plot



Contour plot



Histogram

Built-in functions and toolboxes accelerate and simplify workflow

➤ Built-in functions

- Mathematical: `sin`, `cos`, `exp`, `sqrt`
- Data manipulation: `sort`, `mean`, `std`, `reshape`
- Linear algebra: `inv`, `eig`, `det`

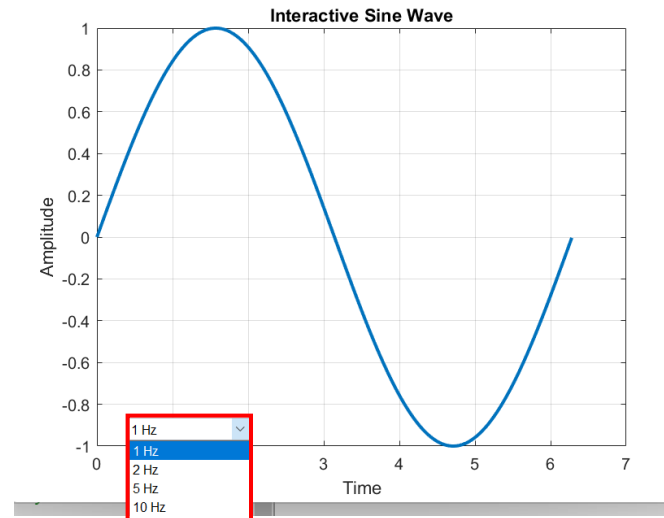
➤ Toolboxes for a variety of applications

Toolboxes					
Engineering	Data science/machine learning	Image/Video processing	Finance and economics	Healthcare and biomedical research	Physics and chemistry
Control systems	Statistics and machine learning	Image processing	Financial	Bioinformatics	Optimization
Signal processing	Deep learning	Computer vision	Econometrics	Biomedical signal processing	Simscape
Simulink	Parallel computing	Video and image processing blockset	Risk management	SimBiology	Curve fitting

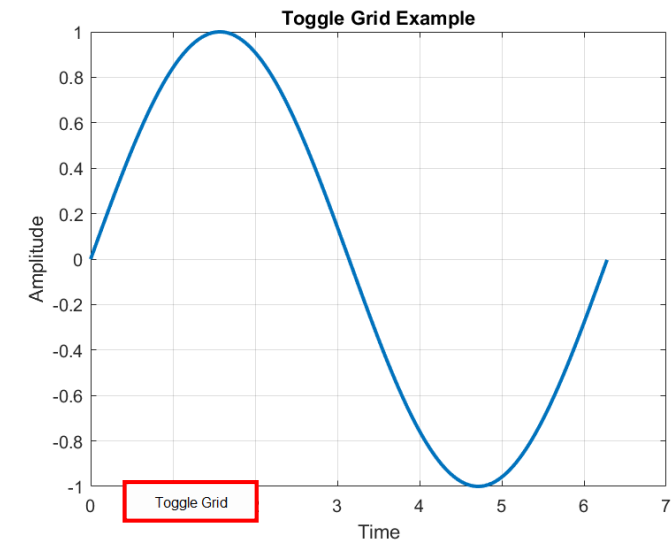
Other toolboxes

- Geospatial analysis
- Transportation
- Social science
- Energy/environment
- Security/Cryptography

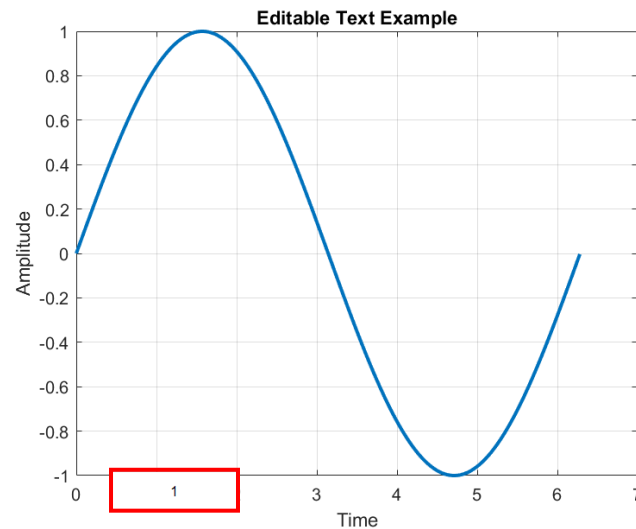
Interactive environment for iterative exploration



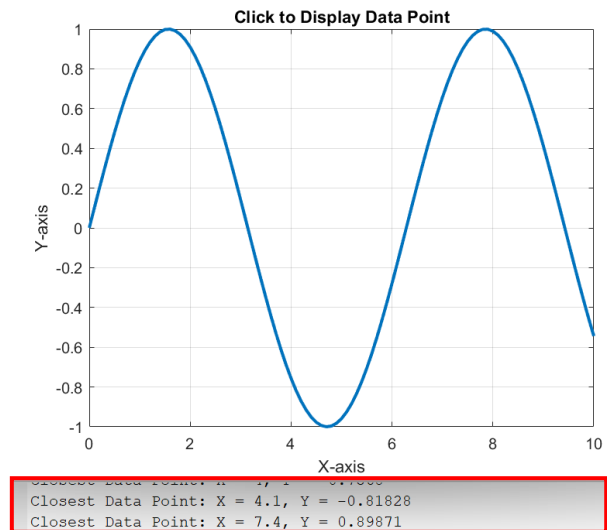
Drop-down menu



Clickable buttons



Editable text fields



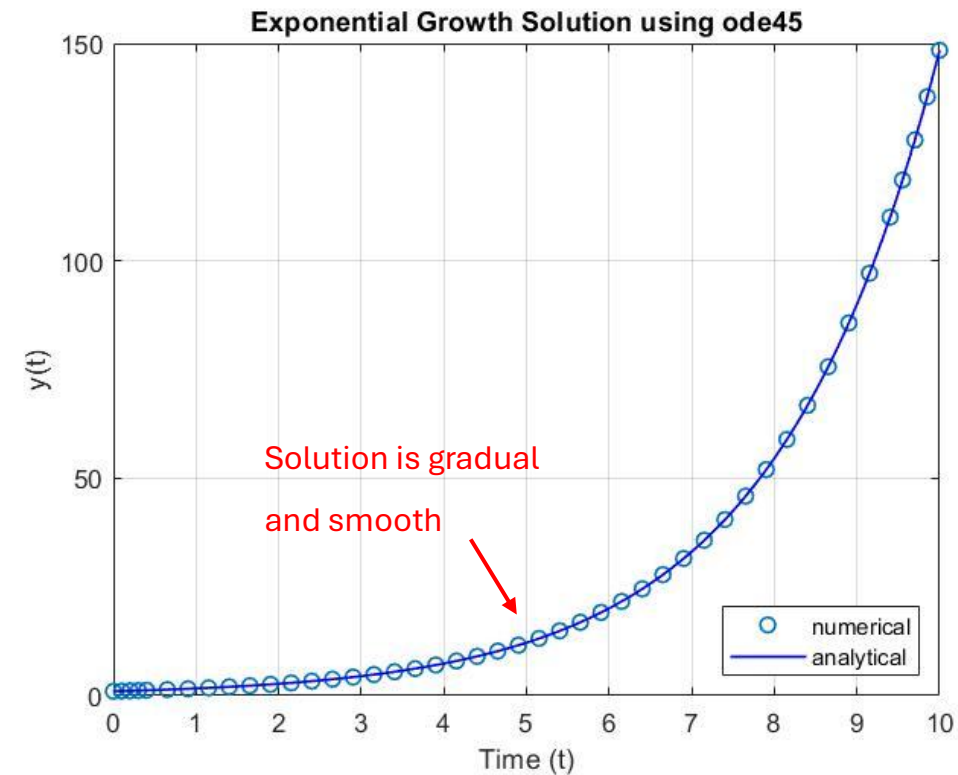
Mouse-click interaction

Solving ordinary differential equations (ODEs) using *ode45*

ode45 is used to solve moderately-sized, **non-stiff ODE** characterized by smooth and gradual changes in the solution over time. Step size by numerical solvers can be relatively large.

Example of non-stiff ODE (first-order initial value problem):

$$\frac{dy}{dt} = 0.5y \quad \text{where } y(0) = 1$$
$$y = e^{0.5t}$$



Class activity – MATLAB code for solving ODE using *ode45*

Equation

$$\frac{dy}{dt} = 0.5y \quad \text{where } y(0) = 1$$

$$y = e^{0.5t}$$

Code

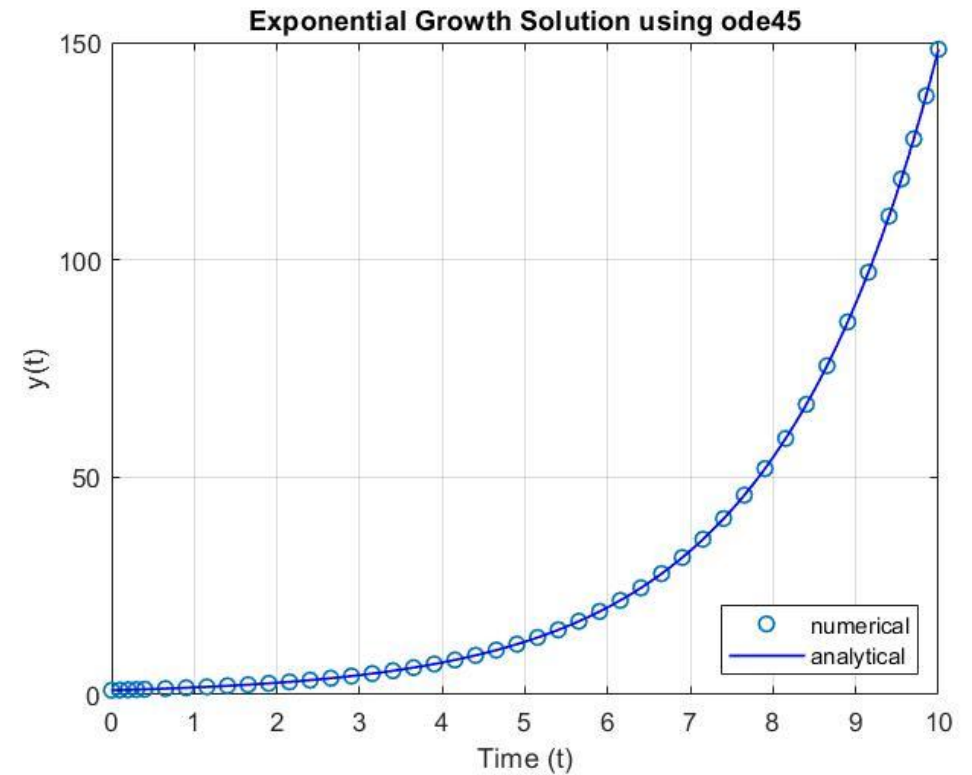
```
% Parameters
r = 0.5; % Growth rate
y0 = 1; % Initial condition
tspan = [0 10]; % Time span for the solution

% Function
growthODE = @(t, y) r * y;

% Solve
[t, y] = ode45(growthODE, tspan, y0); % ode45(function, timeSpan, initialCondition)

% Plot
figure;
plot(t, y, 'b-', 'LineWidth', 1);
xlabel('time/s');
ylabel('y(t)');
grid on;
```

Plot

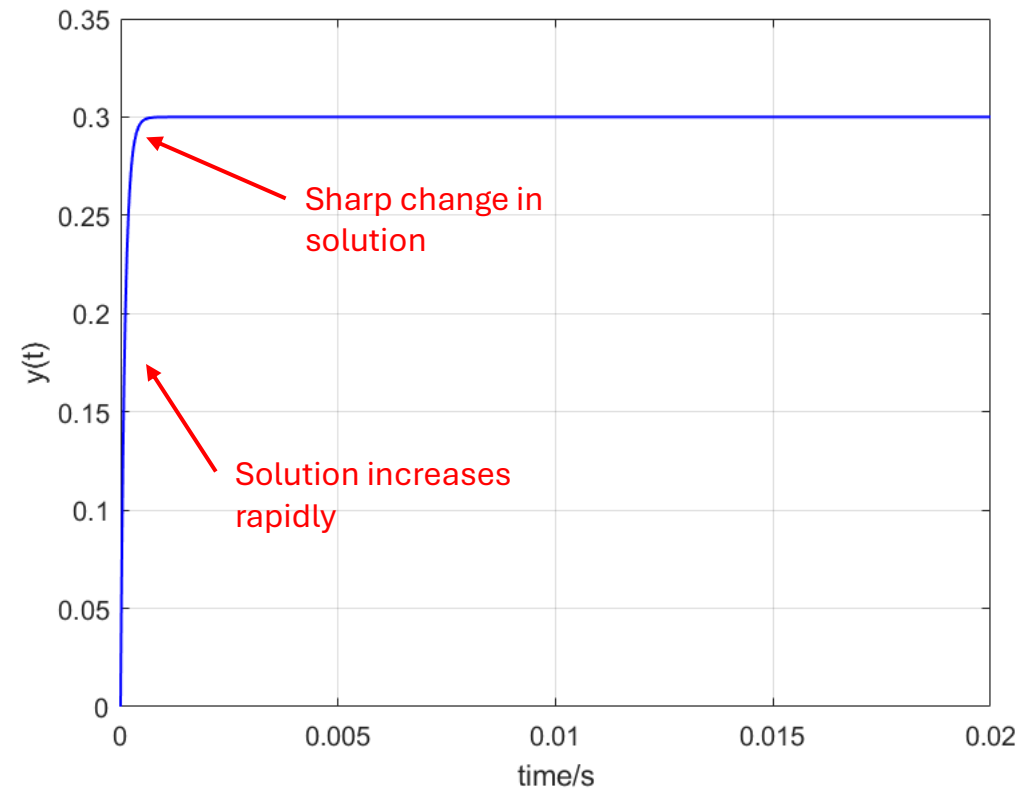


Solving ODEs using *ode15S*

ode15s is used to solve **stiff ODE**, characterized by solutions with very large change in timescales. There are fast changing and slow changing components in the solution and require small steps to maintain stability.

Example of stiff ODE:

$$\frac{dy}{dt} = -10000y + 3000 \quad \text{where } y(0) = 0$$
$$y = 0.3(1 - e^{-10000t})$$



Class activity – MATLAB code for solving ODE using *ode15s*

Equation

$$\frac{dy}{dt} = -10000y + 3000 \quad \text{where } y(0) = 0$$

$$y = 0.3(1 - e^{-10000t})$$

Code

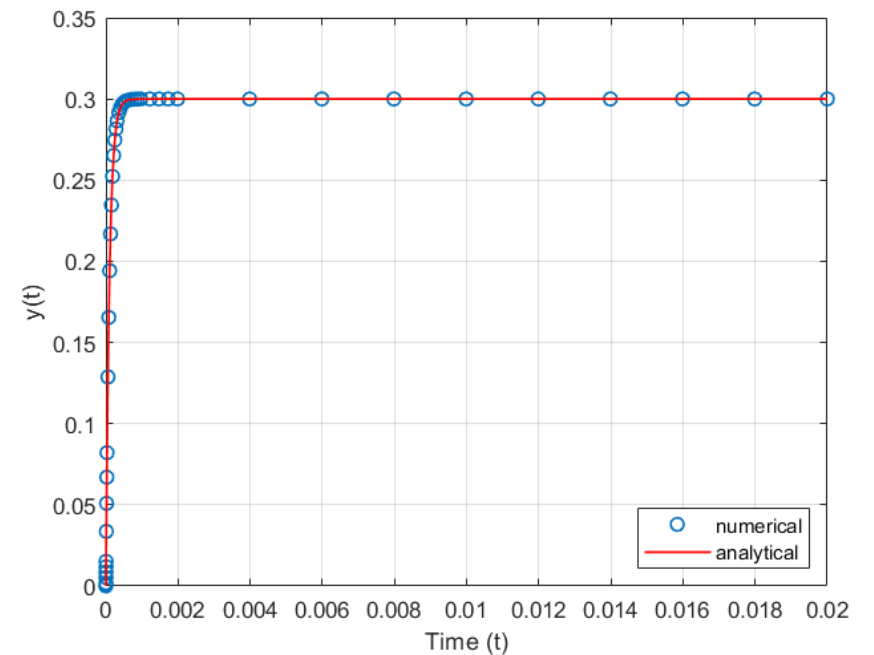
```
% Parameters
k = 1e4;
y0 = 0;
tspan = [0 0.02];

% Function
stiffODE = @(t, y) -k * y + 3000;

% Use ode15s to solve the ODE
[t_ode15s, y_ode15s] = ode15s(stiffODE, tspan, y0); % ode15s(function, timeSpan, initialCondition)

% Plot the results
figure;
plot(t_ode15s, y_ode15s, 'b-', 'LineWidth', 1);
xlabel('time/s');
ylabel('y(t)');
grid on;
```

Plot



Solve an increasingly stiff equation using *ode45* and *ode15s* to compare solver times

```
% Parameters
k = 1e2;
y0 = 0;
tspan = [0 0.02];

% Function
stiffODE = @(t, y) -k * y + 3000;

% Use ode15s to solve the ODE
tic;
[t_ode15s, y_ode15s] = ode15s(stiffODE, tspan, y0);
ode15s_elapsed = toc;

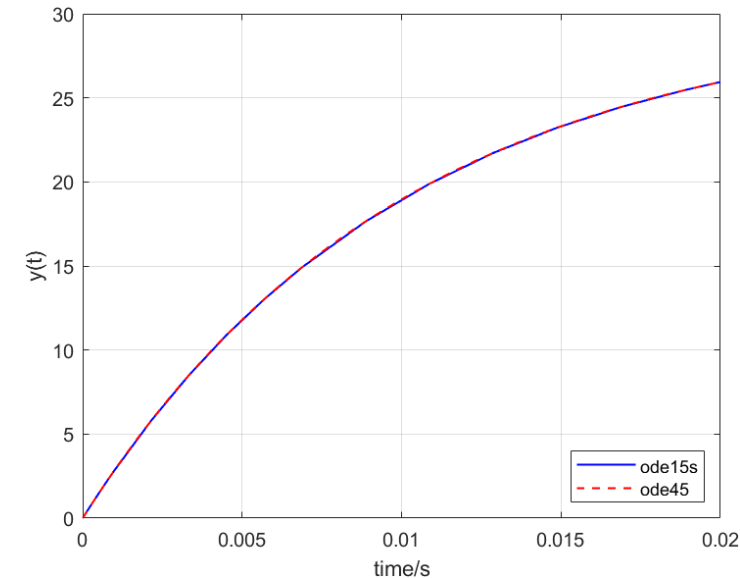
% Use ode45 to solve the ODE
tic;
[t_ode45, y_ode45] = ode45(stiffODE, tspan, y0);
ode45_elapsed = toc;

% Plot the results
figure;
plot(t_ode15s, y_ode15s, 'b-', 'LineWidth', 1);
hold on;
plot(t_ode45, y_ode45, 'r--', 'LineWidth', 1);
hold off;
xlabel('time/s');
ylabel('y(t)');
legend('ode15s', 'ode45', 'Location', 'southeast');
grid on;
disp(['Elapsed time for ode15s: ' num2str(ode15s_elapsed)]);
disp(['Elapsed time for ode45: ' num2str(ode45_elapsed)]);
```

As k increases, the equation becomes increasingly stiff. Solve the equation using k with increasing orders of magnitude, i.e.

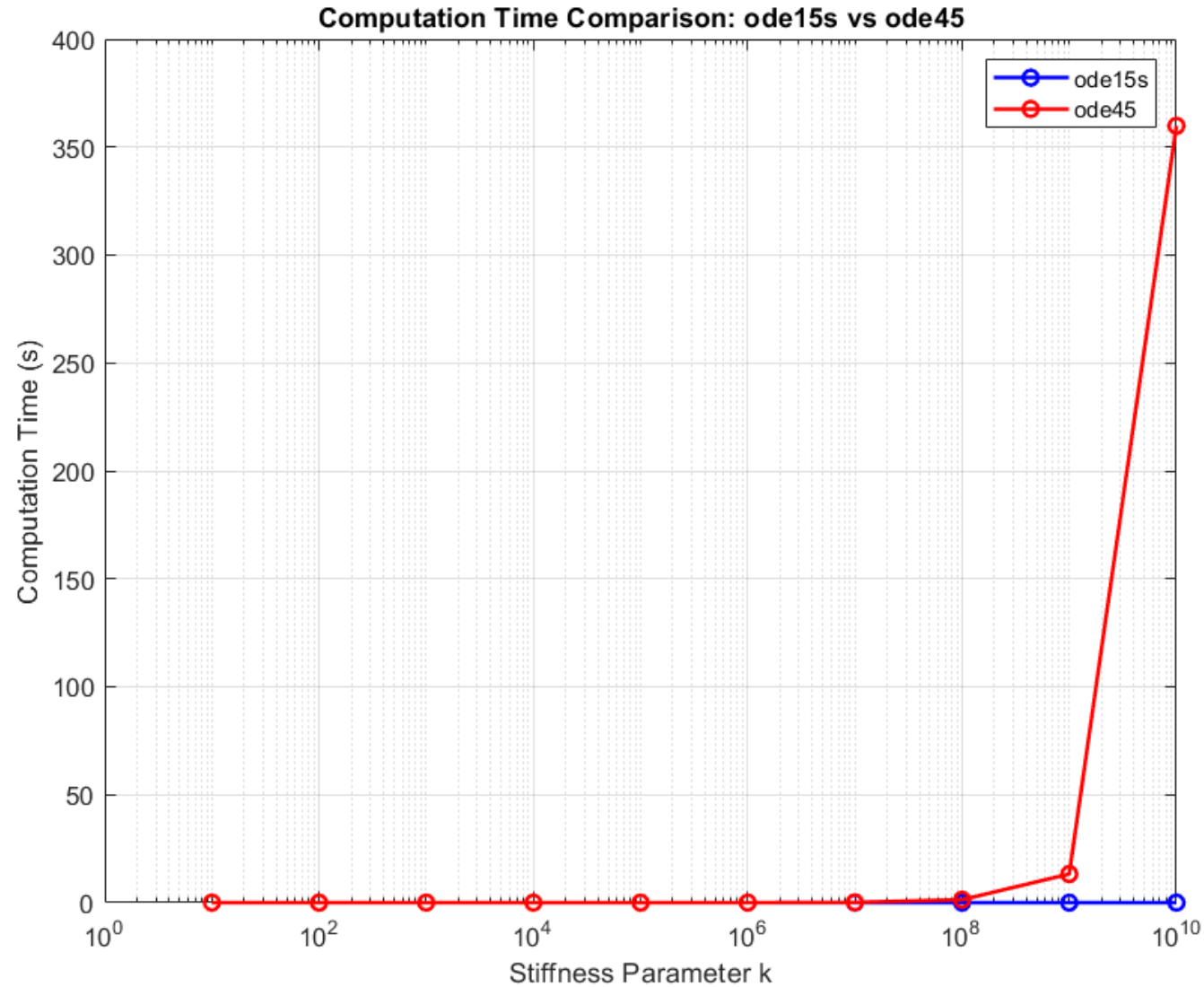
$k = 1e2, 1e3, \dots, 1e7$

What do you notice about the time taken by ode45 and ode15s to solve the equations respectively?



Elapsed time for ode15s: 0.0079064
Elapsed time for ode45: 0.0020994

Computational times increase more quickly when using *ode45* as compared to *ode15s* for increasingly stiff equations



Order of convergency a relation to how quickly numerical solution error reduces when step size decreases

For sufficiently small step size h :

$$Error \propto h^p$$

where p refers to the order of the numerical method. Reducing h reduces the error, but increasing p speeds up the convergence.

Accuracy and efficiency are tradeoffs. To obtain higher accuracy (lower error) necessitates reduction in step size h but requires more time/memory/computational power to execute. Choosing a higher order method requires more work per step but requires fewer total steps to reach target accuracy.

Goal is to achieve desired accuracy while minimizing computational resource

ode45 takes tiny steps for stiff problems, slowing computation compared to *ode15s*

***ode45* uses an explicit Runge-Kutta method with order 5.** The solver automatically adjusts the step size h to meet relative and absolute tolerance.

***ode15s* uses an implicit Backwards Differentiation Formula and can switch between orders 1 to 5 automatically.** Designed for problems where explicit methods like *ode45* need extremely small step sizes for stability.

Thus, we observe that for a very stiff problem *ode45* takes longer to converge to a solution than *ode15s* since the extremely small step sizes for *ode45* results in longer computational times.

Three main types of boundary conditions in boundary value problems

Boundary value problems are differential equation problems where the solution of the equation is sought over a specific domain or interval. As compared to initial-value problems which we have solved earlier where we provide initial conditions, here we provide the *boundary conditions*. There are three types of boundary conditions:

Dirichlet boundary conditions

$$y(a) = \alpha, \quad y(b) = \beta$$

Neumann boundary conditions

$$\frac{dy}{dx} \Big|_{x=a} = \alpha, \quad \frac{dy}{dx} \Big|_{x=b} = \beta$$

Robin (mixed) boundary conditions

$$ay(a) + b \frac{dy}{dx}(b) = \gamma$$

Solving boundary value problems using *bvp4c* by converting second-order ODE into a system of first-order equations

Consider the following boundary value problem

$$\frac{d^2y}{dx^2} + y = 0, \quad y(0) = 1, \quad y\left(\frac{\pi}{2}\right) = 1$$

We convert the second-order ODE into a system of first-order equations by defining two variables y_1 and y_2

$$y_1 = y$$

$$y_2 = y'$$

We express the second-order ODE in terms of the new variables

$$y_1' = y_2$$

$$y_2' = -y_1$$

Class activity – MATLAB code for solving ODE using *bvp4c*

Equation

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= -y_1 \end{aligned} \quad y_1(0) = 1, \quad y_1\left(\frac{\pi}{2}\right) = 1$$

What happens to the boundary condition code if have the following?

$$y'(0) = 1, \quad y(\pi) = 1$$

Code

```
% Define the ODE as a function handle
ode = @(x, y) [y(2); -y(1)]; % System of first-order equations

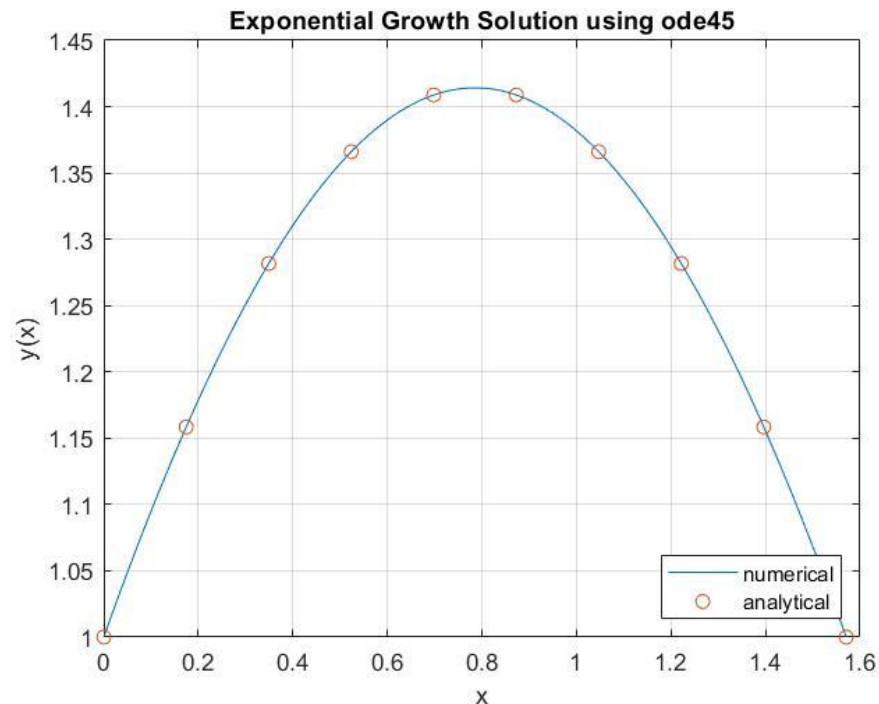
% Define the boundary conditions as a function handle
bc = @(ya, yb) [ya(1) - 1; yb(1) - 1];

% Initial guess for the solution
solinit = bvpinit(linspace(0, 0.5 * pi, 100), [1, 0]);

% Solve the BVP using bvp4c
sol = bvp4c(ode, bc, solinit);

% Plot the solution
figure;
plot(x, y(1, :));
xlabel('x');
ylabel('y(x)');
grid on;
```

Plot



Lesson Recapitulation

- Gained familiarity with MATLAB capabilities
- Solved ODEs using MATLAB's built in functions:
 - *ode45* for non stiff initial value problems
 - *ode15s* for stiff initial value problems
 - *bvp4c* for boundary value problems
- Gained understanding of the concept of *order of convergence*

Announcements

- 1st submission of “exam report” by **23:59, 13/03/2026**.
- Oral presentation of the “exam report” on **18/03/2026 (Week 8)**. 1st presentation starts at 14:15.
- 15 minutes for each presentation, followed by 5 minutes of Q &A.
- Please indicate the Week 7 Individual Meeting time slot in the google spreadsheets. (Monday, March 9, or Wednesday, March 11, 2026 at office 1.302.24)

<https://docs.google.com/spreadsheets/d/10sE6iWVvFoSE9Oo5DYvNaOhwu1nqeNufKXq8NkKbHRs/edit?gid=0#gid=0>